

Les bases de Python

Des variables aux decorateurs

Tout ce qu'il faut pour résoudre des problèmes
et comprendre les erreurs.

Cours à imprimer – support d'entraînement

Table des matières

1. Types de variables

Python est *dynamiquement type* : une variable n'a pas de type declare, elle recoit le type de la valeur qu'on lui affecte. Elle est aussi *fortement typee* : "3" + 5 leve une erreur au lieu de deviner.

1.1 Les types de base

```
n = 42          # int : entier de taille illimitée
x = 3.14        # float : nombre a virgule flottante
b = True        # bool : True / False (sous-classe de int)
s = "bonjour"   # str : chaîne de caracteres (immuable)
z = None        # NoneType : absence de valeur
c = 2 + 3j      # complex

print(type(n))    # <class 'int'>
print(isinstance(b, int)) # True : bool herite de int
```

Chaque valeur en Python est un *objet* qui connaît son propre type. Le type determine les operations autorisees : on peut multiplier deux `int`, mais pas additionner un `int` et une `str`. Retenir la difference entre type *mutable* (modifiable apres creation : liste, dict, set) et type *immuable* (fige : int, float, str, tuple) est essentiel : cela explique beaucoup d'erreurs et de comportements surprenants que l'on verra plus loin.

1.2 Conversions (casting)

Convertir, c'est demander explicitement a Python de fabriquer un nouvel objet d'un autre type a partir d'une valeur. C'est indispensable, par exemple, quand on lit une donnee au clavier (`input` renvoie toujours une `str`) et qu'on veut faire un calcul dessus. Attention : une conversion peut echouer et lever une exception (`int("abc") → ValueError`).

```
int("42")       # 42      (str -> int)
float("3.14")   # 3.14
str(42)         # "42"
int(3.9)        # 3       (troncature, pas d'arrondi)
bool(0)         # False ; bool("") -> False ; bool([]) -> False
```

Piege courant

La division / renvoie toujours un `float` (`6 / 2 == 3.0`). Pour un entier, utilise la division entiere `//` (`7 // 2 == 3`).

2. Operateurs

Les operateurs sont les symboles qui combinent ou comparent des valeurs. Deux familles reviennent sans cesse dans la resolution de problemes : les operateurs de *comparaison* (qui renvoient un booleen `True/False` et pilotent les conditions) et les operateurs *logiques* (`and`, `or`, `not`) qui les combinent. Bon a savoir : `and` et `or` sont *paresseux* (court-circuit) – `a and b` n'evalue `b` que si `a` est vrai, ce qui permet d'ecrire des tests surs comme `if x and x[0] == 1`.

Categorie	Operateurs
Arithmetiques	+ - * / // % **
Comparaison	== != < > <= >=
Logiques	and or not
Appartenance	in , not in

Identite	<code>is , is not</code>
Affectation	<code>= += -= *= /= //=</code>

Piege courant

`==` compare les *valeurs* ; `is` compare l'*identite* (meme objet en memoire). Utilise `is` uniquement avec `None` : `if x is None`.

3. Chaines de caracteres

Une chaine (`str`) est une suite ordonnee de caracteres. On y accede par *indexation* (`s[i]`, le premier caractere est a l'indice 0) et par *tranche* (`s[debut:fin]`, la borne de fin etant exclue). Les indices negatifs comptent depuis la fin, ce qui evite de calculer la longueur. Comme les chaines sont **immuables**, toutes les methodes qui « modifient » une chaine (comme `upper` ou `replace`) renvoient en realite une *nouvelle* chaine : l'originale reste intacte. Les *f-strings* (`f"..."`) sont la facon moderne et lisible d'insérer des variables dans un texte.

```
s = "Python"
len(s)           # 6
s[0]             # 'P'           (indexation)
s[-1]            # 'n'           (indice negatif = depuis la fin)
s[0:3]           # 'Pyt'        (slicing [debut:fin])
s.upper()        # 'PYTHON'
s.replace("Py", "Ja") # 'Jathon'
", ".join(["a", "b"]) # 'a, b'
"a,b,c".split(",")  # ['a', 'b', 'c']

nom, age = "Alice", 30
f"{nom} a {age} ans" # f-string : 'Alice a 30 ans'
```

Piege courant

Les chaines sont *immuables* : `s[0] = "X"` leve `TypeError`. Il faut reconstruire une nouvelle chaine.

3.1 Astuces de manipulation

Au-dela des bases, une poignee de methodes reviennent sans cesse des qu'on doit *nettoyer*, *aligner* ou *analyser* du texte. Les connaitre evite de reecrire a la main des boucles que la bibliotheque standard fait deja tres bien.

```
# Nettoyage : strip enleve les caracteres indesirables aux extremités
"  bonjour ".strip()      # 'bonjour'   (espaces des deux cotes)
"##titre##".strip("#")    # 'titre'     (caractere precis)
"  bonjour".lstrip()       # 'bonjour'   (seulement a gauche)
"bonjour ".rstrip()       # 'bonjour'   (seulement a droite)

# Alignement / padding : utile pour formater un affichage tabulaire
"7".zfill(3)               # '007'      (complete avec des zeros a gauche)
"7".rjust(3, "0")          # '007'      (equivalent, plus general)
"ok".ljust(5, ".")         # 'ok...'    (aligne a gauche)
"ok".center(6, "-")        # '-ok--'    (centre)

# Recherche et tests
"algorithme".startswith("algo") # True
"algorithme".endswith("me")     # True
"algorithme".find("go")         # 2         (-1 si absent, ne leve pas d'erreur)
```

```

"algorithme".index("go")          # 2 (leve ValueError si absent)
"abc123".isalpha()                 # False (contient des chiffres)
"abc123".isalnum()                 # True (lettres + chiffres uniquement)

# Remplacement multiple d'un coup avec translate (rapide, sans boucle)
table = str.maketrans("ae", "AE")  # a->A, e->E
"algorithme".translate(table)      # 'AlgorithmE'

# split avec limite, et rsplit pour couper depuis la droite
"a=b=c".split("=", 1)              # ['a', 'b=c'] (une seule coupe)
"a/b/c.txt".rsplit("/", 1)         # ['a/b', 'c.txt'] (coupe depuis la fin)
" a b c ".split()                  # ['a', 'b', 'c'] (split() sans argument)
                                   # ignore les espaces multiples/en trop

# f-strings avancees : formater sans concatener
prix = 1234.5
f"{prix:,.2f}"                     # '1,234.50' (separateur de milliers, 2 decimales)
f"{prix:>10.1f}"                   # ' 1234.5' (aligne a droite sur 10 caracteres)
nom = "Alice"
f"{nom=}"                          # "nom='Alice'" (affiche aussi le nom de variable, debug
                                   rapide)

```

Compter des caracteres : le reflexe Counter

Plutôt que d'écrire une boucle avec un dictionnaire pour compter les occurrences de chaque caractère, `collections.Counter` le fait en une ligne et offre directement les caractères les plus fréquents.

```

from collections import Counter

freq = Counter("mississippi")      # Counter({'i': 4, 's': 4, 'p': 2, 'm': 1})
freq.most_common(2)                # [('i', 4), ('s', 4)] (les 2 plus frequents)
freq["i"]                          # 4 (0 si absent, jamais de KeyError)

```

Piège courant

"a" in "banane" teste une *sous-chaine*, pas un caractère isolé uniquement : "an" in "banane" vaut aussi True. Pratique, mais ne pas confondre avec un test d'égalité caractère par caractère.

4. Structures de données

Choisir la bonne structure de données est souvent la moitié de la solution d'un problème. Python en offre quatre fondamentales, qu'on distingue selon trois questions : les éléments sont-ils *ordonnés* ? la collection est-elle *modifiable* ? les accès se font-ils par position ou par cle ? Un bon réflexe : si tu écris une boucle pour vérifier « est-ce que x est dans la liste ? » de façon répétée, un `set` ou un `dict` (recherche en $O(1)$ moyen) remplace avantageusement la liste (recherche en $O(n)$).

4.1 Liste – ordonnée, modifiable

La liste est la structure à tout faire : une séquence ordonnée que l'on peut agrandir, trier et modifier sur place. Les *compréhensions* de liste sont une façon concise et rapide de construire une liste à partir d'une autre séquence, en remplaçant une boucle `for` et des `append`.

```

nums = [3, 1, 2]
nums.append(4)      # [3, 1, 2, 4]
nums.sort()         # [1, 2, 3, 4] (sur place)

```

```
nums[1:3]          # [2, 3]          (slicing)
carres = [x*x for x in range(5)]    # comprehension : [0,1,4,9,16]
```

4.2 Tuple – ordonne, immuable

Le tuple ressemble a une liste mais ne peut plus changer une fois cree. On l'utilise pour des donnees qui vont ensemble et ne doivent pas bouger (un point (**x**, **y**), une ligne d'une base de donnees) et pour le *deballage* (unpacking) qui affecte plusieurs variables en une ligne. Etant immuable, un tuple peut servir de cle de dictionnaire, contrairement a une liste.

```
point = (3, 4)
x, y = point          # deballage (unpacking)
# point[0] = 9         -> TypeError : un tuple ne se modifie pas
```

4.3 Dictionnaire – cle -> valeur

Le dictionnaire associe des *cles* a des *valeurs* et permet de retrouver une valeur instantanement a partir de sa cle. C'est l'outil ideal pour compter, indexer, ou représenter un objet par ses attributs. La methode `get(cle, default)` evite un `KeyError` lorsque la cle peut etre absente. Depuis Python 3.7, un dictionnaire conserve l'ordre d'insertion des cles.

```
age = {"Alice": 30, "Bob": 25}
age["Alice"]          # 30
age.get("Zoe", 0)     # 0 (valeur par default si absente)
age["Zoe"] = 22       # ajout
for cle, val in age.items():
    print(cle, val)
```

Astuces de manipulation

Le dictionnaire est la structure la plus polyvalente de Python : elle sert a la fois de *table de correspondance*, de *compteur* et d'*index*. Quelques idiomes reviennent dans presque tous les problemes d'algorithmique.

```
# Dict comprehension : construire un dict en une ligne, comme pour une liste
carres = {x: x*x for x in range(5)}          # {0:0, 1:1, 2:4, 3:9, 4:16}
mots = ["chat", "chien", "oiseau"]
longueurs = {m: len(m) for m in mots}        # {'chat':4, 'chien':5, 'oiseau':7}

#.setdefault : recupere une cle, ou l'insere avec une valeur par default
graphe = {}
graphe.setdefault("A", []).append("B")       # cree [] si absent, puis append
graphe.setdefault("A", []).append("C")       # {'A': ['B', 'C']}

# defaultdict : evite setdefault repete, la valeur par default est automatique
from collections import defaultdict
groupes = defaultdict(list)
for mot in ["chat", "chien", "chameau"]:
    groupes[mot[0]].append(mot)               # pas besoin de tester si 'c' existe
# {'c': ['chat', 'chien', 'chameau']}

# Fusionner deux dicts (Python 3.9+) : | cree un nouveau dict, |= modifie en place
defaults = {"theme": "clair", "langue": "fr"}
utilisateur = {"langue": "en"}
config = defaults | utilisateur              # {'theme':'clair', 'langue':'en'}
```

```

# (les cle de droite ecrasent celles de
gauche)

# Parcourir cle / valeurs / paires
list(age.keys())      # ['Alice', 'Bob', 'Zoe']
list(age.values())    # [30, 25, 22]
list(age.items())     # [('Alice', 30), ('Bob', 25), ('Zoe', 22)]

# Trier un dict par valeur (frequent : classement par score, par frequence...)
scores = {"Alice": 90, "Bob": 75, "Zoe": 88}
sorted(scores.items(), key=lambda kv: kv[1], reverse=True)
# [('Alice', 90), ('Zoe', 88), ('Bob', 75)] (trie decroissant sur la valeur)

# Inverser un dict (valeur -> cle), valable seulement si les valeurs sont uniques
{v: k for k, v in age.items()}

# Supprimer une cle sans planter si elle est absente
age.pop("Inconnu", None)      # None si absente ; pas de KeyError

```

Piege courant

Une cle de dictionnaire doit etre *immuable* (hashable) : `str`, `int`, `tuple` conviennent, mais une `list` comme cle leve `TypeError: unhashable type`. Utilise un `tuple` si tu as besoin d'une cle composee, par exemple `grille[(x, y)] = valeur`.

4.4 Ensemble (set) – valeurs uniques

Un ensemble ne contient jamais de doublon et n'a pas d'ordre. On l'emploie pour eliminer les repetitions (`set(ma_liste)`), tester tres vite une appartenance, ou faire des operations mathematiques : union (`|`), intersection (`&`), difference (`-`). C'est souvent l'astuce qui transforme un algorithme lent en algorithme rapide.

```

vus = {1, 2, 2, 3}      # {1, 2, 3}
vus.add(4)
3 in vus                # True  (appartenance en O(1) moyen)
{1, 2} & {2, 3}         # {2}   (intersection)

```

Quel type choisir ?

Ordre + modifications → **liste**. Donnees fixes → **tuple**. Correspondance cle/valeur → **dict**.
Unicite et appartenance rapide → **set**.

5. Conditions et boucles

Les conditions (`if/elif/else`) et les boucles (`for, while`) controlent le *flux* d'execution : quelles instructions s'executent, et combien de fois. En Python, les blocs ne sont pas delimites par des accolades mais par l'**indentation** : tout ce qui est decale d'un cran vers la droite appartient au bloc. Une indentation incoherente provoque une `IndentationError`. La boucle `for` parcourt directement les elements d'une sequence (pas des indices); `enumerate` fournit en prime la position, et `range` genere une suite d'entiers.

```

# Condition
if n > 0:
    signe = "positif"
elif n == 0:
    signe = "nul"

```

```

else:
    signe = "negatif"

# Boucle for
for i in range(3):          # 0, 1, 2
    print(i)

for i, val in enumerate(["a", "b"]): # (0,'a') (1,'b')
    print(i, val)

# Boucle while
while n > 0:
    n -= 1

# break / continue / else
for x in nums:
    if x == cible:
        break                # sort de la boucle
else:
    print("non trouve")      # execute si pas de break

```

Piege courant

`range(a, b)` s'arrete a `b-1` (borne haute exclue). `range(5)` produit 0,1,2,3,4.

6. Fonctions

Une fonction regroupe un morceau de code reutilisable sous un nom. Elle prend des *parametres* en entree et renvoie un resultat avec `return` (sans `return`, elle renvoie `None`). Bien decouper un programme en petites fonctions au role unique le rend plus lisible, plus facile a tester et a deboguer. Python offre des arguments *par default*, des arguments *nommes* (qui clarifient les appels), et la syntaxe `*args` / `**kwargs` pour accepter un nombre variable d'arguments.

```

def salue(nom, salutation="Bonjour"): # argument par default
    return f"{salutation}, {nom} !"

salue("Alice")                        # 'Bonjour, Alice !'
salue("Bob", "Salut")                 # 'Salut, Bob !'
salue(nom="Zoe")                      # argument nomme

# Nombre variable d'arguments
def total(*args, **kwargs):           # args = tuple, kwargs = dict
    return sum(args)

total(1, 2, 3)                        # 6

# Fonction anonyme (lambda)
carre = lambda x: x * x
sorted(mots, key=lambda m: len(m))    # trier par longueur

```

Piege courant

Ne jamais utiliser un objet *mutable* comme valeur par default : `def f(x, liste=[])` partage la meme liste entre les appels. Ecris `def f(x, liste=None): liste = liste or []`.

7. Portee des variables (scope)

```
compteur = 0                                # variable globale

def incr():
    global compteur                        # sinon : variable locale
    compteur += 1

def exterieure():
    x = 10
    def interieure():
        nonlocal x                        # modifie x de la fonction englobante
        x += 1
    interieure()
    return x                                # 11
```

Regle LEGB

Python cherche un nom dans cet ordre : **L**ocal → **E**nglobant → **G**lobal → **B**uilt-in.

8. Classes et objets

Une *classe* est un modele (un plan) qui decrit des donnees (les *attributs*) et des comportements (les *methodes*). Un *objet* est une instance concrete de ce modele. Le premier parametre de chaque methode, **self**, designe l'objet sur lequel on travaille ; la methode speciale `__init__` est le *constructeur*, appele automatiquement a la creation. La *programmation orientee objet* permet de regrouper l'etat et le comportement au meme endroit, et l'*heritage* (via `super()`) de reutiliser et specialiser une classe existante sans la reecrire.

```
class Compte:
    taux = 0.02                            # attribut de classe (partage)

    def __init__(self, solde):              # constructeur
        self.solde = solde                 # attribut d'instance

    def deposer(self, montant):             # methode
        self.solde += montant

    def __str__(self):                      # representation lisible
        return f"Compte({self.solde} EUR)"

c = Compte(100)
c.deposer(50)
print(c)                                  # Compte(150 EUR)

# Heritage
class ComptePremium(Compte):
    def __init__(self, solde, bonus):
        super().__init__(solde)           # appelle le parent
        self.bonus = bonus
```

Piege courant

Oublier **self** en premier parametre des methodes leve `TypeError: ... takes 0 positional arguments but 1 was given.`

9. Gestion des erreurs (exceptions)

Une erreur non gérée arrête le programme et affiche une *traceback*. Plutôt que de la subir, on peut l'*anticiper* : le bloc `try` contient le code risqué, et chaque `except` rattrape un type d'erreur précis pour réagir proprement. C'est la différence entre un programme qui plante brutalement et un programme robuste qui affiche un message clair et continue. Le bloc `else` s'exécute si aucune erreur n'est survenue, et `finally` s'exécute *toujours* (succès ou échec) – idéal pour libérer une ressource, fermer un fichier ou une connexion. On peut aussi *déclencher* soi-même une erreur avec `raise` quand une donnée est invalide.

```
try:
    x = int(input("Nombre : "))
    resultat = 10 / x
except ValueError:                # conversion impossible
    print("Ce n'est pas un entier.")
except ZeroDivisionError:        # division par zero
    print("Division par zero interdite.")
except Exception as e:           # filet de securite
    print(f"Erreur inattendue : {e}")
else:
    print("Tout s'est bien passe :", resultat)
finally:
    print("Toujours execute (nettoyage).")

# Lever sa propre exception
def racine(x):
    if x < 0:
        raise ValueError("x doit etre positif")
    return x ** 0.5
```

Piège courant

Évitez `except` : tout seul (attrape même `Ctrl+C`). Attrape toujours le type le plus précis possible, et place les `except` du plus spécifique au plus général.

10. Les erreurs Python les plus fréquentes

Savoir *lire* une erreur fait gagner un temps précieux : le nom de l'exception dit *quel genre* de problème s'est produit, et le message dit *où* et *pourquoi*. Voici les exceptions que tu rencontreras le plus souvent en débutant. Prends l'habitude d'associer chaque nom à sa cause typique : avec un peu de pratique, tu diagnostiqueras la plupart des bugs en lisant simplement la première ligne du message.

Exception	Cause typique
<code>SyntaxError</code>	Faute de syntaxe (: oubliée, parenthèse non fermée).
<code>IndentationError</code>	Indentation incorrecte ou incohérente (espaces vs tabulations).
<code>NameError</code>	Variable utilisée avant d'être définie.
<code>TypeError</code>	Opération entre types incompatibles (" <code>a</code> " + 1).
<code>ValueError</code>	Bon type mais valeur invalide (<code>int("abc")</code>).
<code>IndexError</code>	Indice hors des bornes d'une liste.
<code>KeyError</code>	Cle absente d'un dictionnaire.

<code>AttributeError</code>	Attribut/methode inexistant sur un objet.
<code>ZeroDivisionError</code>	Division ou modulo par zero.
<code>ImportError</code>	Module ou nom introuvable a l'import.
<code>RecursionError</code>	Recursion trop profonde (cas de base manquant).

Lire une traceback

Lis-la **de bas en haut** : la dernière ligne donne le type d'erreur et le message ; remonte pour trouver le fichier et la ligne exacte où elle s'est produite.

11. Modules et imports

Un *module* est un fichier Python que l'on peut réutiliser dans un autre grâce à `import`. La *bibliothèque standard* fournit déjà des centaines de modules prêts à l'emploi (`math`, `collections`, `datetime`, `random`...) – inutile de reinventer la roue. On peut importer un module entier, seulement certains noms (`from ... import ...`), ou lui donner un alias plus court (`import numpy as np`). Le test `if __name__ == "__main__":` permet qu'un fichier serve à la fois de programme exécutable et de module importable.

```
import math
math.sqrt(16)                                # 4.0

from collections import Counter, defaultdict
Counter("aab")                               # Counter({'a': 2, 'b': 1})

import datetime as dt                        # alias
dt.date.today()

# Point d'entree d'un script
if __name__ == "__main__":
    main()                                   # execute seulement si lance directement
```

12. Iterateurs et generateurs

Un *iterateur* est un objet que l'on peut parcourir élément par élément avec une boucle `for`. Un *generateur* est la façon la plus simple d'en créer un : c'est une fonction qui, au lieu de `return`, utilise `yield` pour livrer ses valeurs *une à une*, en se mettant en pause entre chaque. Cette approche *paresseuse* (lazy) ne calcule la valeur suivante qu'au moment où on en a besoin : la mémoire consommée reste constante, même pour parcourir des millions d'éléments ou un flux potentiellement infini.

```
def compte_jusqua(n):
    i = 0
    while i < n:
        yield i                                # renvoie une valeur et se met en pause
        i += 1

for v in compte_jusqua(3):                    # 0, 1, 2
    print(v)

# Expression generatrice (parentheses au lieu de crochets)
somme = sum(x*x for x in range(1000))        # ne cree pas la liste intermediaire
```

Interet

Ideal pour de gros flux de donnees : la memoire reste constante quel que soit le nombre d'elements parcourus.

13. Decorateurs

Un decorateur est une fonction qui *enveloppe* une autre fonction pour lui ajouter un comportement (journalisation, mesure de temps, mise en cache, verification des droits...) **sans toucher a son code**. C'est possible parce qu'en Python les fonctions sont des *objets de premiere classe* : on peut les passer en argument, les renvoyer, et les stocker dans des variables. La syntaxe `@decorateur` placee au-dessus d'une fonction est simplement un raccourci pour `ma_fonction = decorateur(ma_fonction)`. Le decorateur renvoie une fonction *wrapper* qui execute du code avant et/ou apres la fonction d'origine. On l'utilise partout ou l'on veut appliquer le meme traitement a plusieurs fonctions sans dupliquer ce code.

```
import functools
import time

def chronometre(func):
    @functools.wraps(func)           # conserve nom / docstring
    def wrapper(*args, **kwargs):
        debut = time.perf_counter()
        resultat = func(*args, **kwargs)
        duree = time.perf_counter() - debut
        print(f"{func.__name__} a pris {duree:.4f}s")
        return resultat
    return wrapper

@chronometre                        # equivaut a : lent = chronometre(lent)
def lent():
    time.sleep(0.1)

lent()                             # affiche : lent a pris 0.10xx s
```

13.1 Decorateurs integres utiles

```
from functools import lru_cache

@lru_cache(maxsize=None)           # memoise les resultats
def fib(n):
    return n if n < 2 else fib(n-1) + fib(n-2)

class Cercle:
    def __init__(self, r):
        self._r = r

    @property                       # acces comme un attribut
    def aire(self):
        return 3.14159 * self._r ** 2

    @staticmethod                  # pas de self
    def unite():
        return "cm"
```

Piege courant

Sans `@functools.wraps`, la fonction decoree perd son nom et sa docstring (`func.__name__` devient `'wrapper'`), ce qui gene le debogage.

14. Fiche recapitulative

Concept	A retenir
Typage	Dynamique et fort ; <code>type(x)</code> , <code>isinstance(x, T)</code> .
/ vs //	/ = float ; // = division entiere.
Immuable	<code>str</code> , <code>tuple</code> , <code>int</code> ne se modifient pas.
Comprehension	<code>[f(x) for x in it if cond]</code> .
<code>==</code> vs <code>is</code>	valeur vs identite ; <code>is</code> avec <code>None</code> .
Default mutable	Ne jamais mettre [] ou {} en default.
Exceptions	<code>try/except/else/finally</code> ; du specifique au general.
Traceback	Se lit de bas en haut.
Generateur	<code>yield</code> : valeurs paresseuses, memoire constante.
Decorateur	Enveloppe une fonction ; <code>@functools.wraps</code> .
<code>lru_cache</code>	Memoisation automatique des appels.
<code>__main__</code>	<code>if __name__ == "__main__":</code> pour le point d'entree.

Methode pour deboguer un probleme

1. Lire le **type** d'exception et le message (derniere ligne).
2. Localiser le **fichier** et la **ligne** dans la traceback.
3. Afficher les valeurs suspectes (`print` ou `breakpoint()`).
4. Reproduire avec un **petit** exemple minimal.
5. Corriger, puis verifier les **cas limites** (vide, zero, negatif).